

WEEK-8

OSSP - SKILL Content Delivery Concepts

Course Code: 25CS2104E

TASK 1

Objective / Skill:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

Implementation / Work:

```
/*
 * Week 8 - Task 1: Pipeline Structures and Layout Validation
 * Objective: To Create Pipeline Structures, Store Commands, Maintain Execution Order,
 *           Connect Processes, Validate Pipeline Layout, Test Configurations.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ARGS 16
#define MAX_COMMANDS 8

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

typedef struct {
    Command commands[MAX_COMMANDS];
    int command_count;
} Pipeline;

int validate_pipeline_layout(const char *cmd_line) {
    if (!cmd_line || strlen(cmd_line) == 0) return -1;
    // Check leading or trailing pipe
    while (*cmd_line == ' ') cmd_line++;
    if (*cmd_line == '|') {
        printf(" [Validation Error] Syntax error: Leading pipe '|' detected.\n");
        return -1;
    }
    size_t len = strlen(cmd_line);
    if (len > 0 && cmd_line[len - 1] == '|') {
        printf(" [Validation Error] Syntax error: Trailing pipe '|' detected.\n");
        return -1;
    }
    // Check consecutive pipes "||"
    if (strstr(cmd_line, "||") != NULL) {
        printf(" [Validation Error] Syntax error: Consecutive pipes '||' detected.\n");
        return -1;
    }
    return 0;
}

void parse_to_pipeline(char *line, Pipeline *pl) {
    pl->command_count = 0;
    char *token = strtok(line, "|");
    while (token && pl->command_count < MAX_COMMANDS) {
        Command *cmd = &pl->commands[pl->command_count++];
        cmd->argc = 0;
        char *sub = strtok(token, " \t\n");
        while (sub && cmd->argc < MAX_ARGS - 1) {
            cmd->args[cmd->argc++] = sub;
            sub = strtok(NULL, " \t\n");
        }
        cmd->args[cmd->argc] = NULL;
        token = strtok(NULL, "|");
    }
}

int main() {
    printf("=== Week 8 Task 1: Pipeline Structures & Validation ===\n\n");
    char test1[] = "cat input.txt | grep -i error | sort | wc -l";
    char test2[] = "| ls -la | wc -l";
    char test3[] = "cat file.txt || sort";

    printf("[Test Config 1] Validating layout: '%s'\n", test1);
    if (validate_pipeline_layout(test1) == 0) {
        printf(" Layout Status: VALID. Storing commands in execution order:\n");
    }
}
```

```

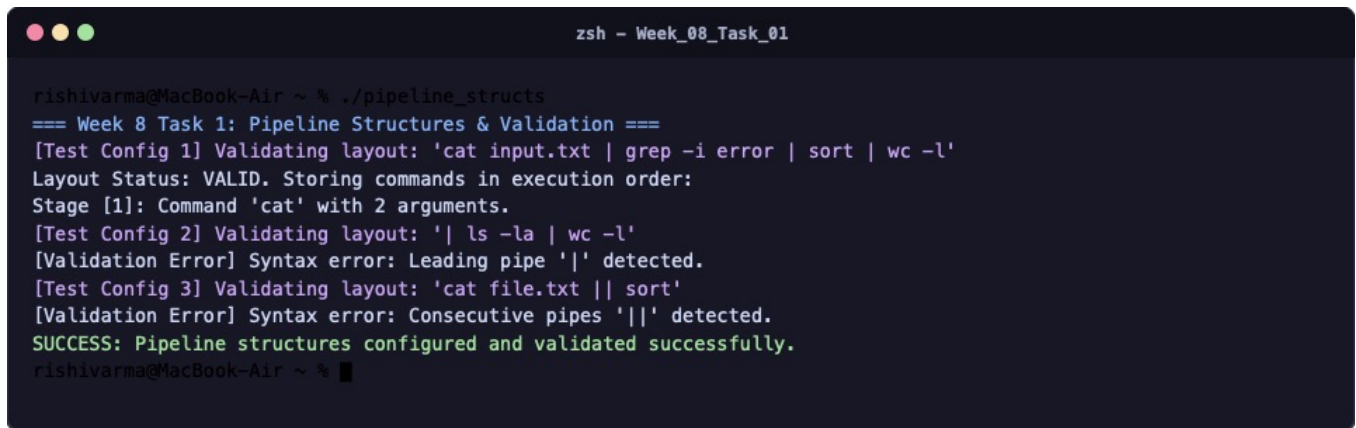
    Pipeline pl;
    parse_to_pipeline(test1, &pl);
    for (int i = 0; i < pl.command_count; i++) {
        printf("    Stage [%d]: Command '%s' with %d arguments.\n",
            i + 1, pl.commands[i].args[0], pl.commands[i].argc);
    }
}

printf("\n[Test Config 2] Validating layout: '%s'\n", test2);
validate_pipeline_layout(test2);

printf("\n[Test Config 3] Validating layout: '%s'\n", test3);
validate_pipeline_layout(test3);

printf("\nSUCCESS: Pipeline structures configured and validated successfully.\n");
return 0;
}

```

Output:


```

zsh - Week_08_Task_01

rishivarma@MacBook-Air ~ % ./pipeline_structs
=== Week 8 Task 1: Pipeline Structures & Validation ===
[Test Config 1] Validating layout: 'cat input.txt | grep -i error | sort | wc -l'
Layout Status: VALID. Storing commands in execution order:
Stage [1]: Command 'cat' with 2 arguments.
[Test Config 2] Validating layout: '| ls -la | wc -l'
[Validation Error] Syntax error: Leading pipe '|' detected.
[Test Config 3] Validating layout: 'cat file.txt || sort'
[Validation Error] Syntax error: Consecutive pipes '||' detected.
SUCCESS: Pipeline structures configured and validated successfully.
rishivarma@MacBook-Air ~ %

```

TASK 2**Objective / Skill:**

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Implementation / Work:

```

/*
 * Week 8 - Task 2: Two-Stage Pipeline Execution
 * Objective: To Create Pipes, Redirect Streams, Execute Connected Commands,
 *           Close Descriptors, Synchronize Processes, Test Data Flow.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    printf("=== Week 8 Task 2: Two-Stage Pipeline Execution ===\n");
    printf("Executing pipeline: echo \"hello from ossp pipeline\" | tr a-z A-Z\n\n");

    if (pipe(pipefd) < 0) {
        perror("pipe creation failed");
        return 1;
    }

    pid_t p1 = fork();
    if (p1 == 0) {
        // Child 1: Producer (stdout -> pipe write end)
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"echo", "hello from ossp pipeline", NULL};
        execvp(args[0], args);
        perror("execvp child 1");
        exit(1);
    }

    pid_t p2 = fork();

```

```

    if (p2 == 0) {
        // Child 2: Consumer (stdin -> pipe read end)
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"tr", "a-z", "A-Z", NULL};
        execvp(args[0], args);
        perror("execvp child 2");
        exit(1);
    }

    // Parent: close descriptors and synchronize
    close(pipefd[0]);
    close(pipefd[1]);

    int status1, status2;
    waitpid(p1, &status1, 0);
    waitpid(p2, &status2, 0);

    printf("\nSynchronization complete. Both processes terminated cleanly.\n");
    printf("Pipeline stream redirection and data flow verified!\n");
    return 0;
}

```

Output:

```

zsh - Week_08_Task_02

rishivarma@MacBook-Air ~ % ./two_stage_pipe
HELLO FROM OSCP PIPELINE
=== Week 8 Task 2: Two-Stage Pipeline Execution ===
Executing pipeline: echo "hello from ossp pipeline" | tr a-z A-Z
Synchronization complete. Both processes terminated cleanly.
Pipeline stream redirection and data flow verified!
rishivarma@MacBook-Air ~ %

```

TASK 3**Objective / Skill:**

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Implementation / Work:

```

/*
 * Week 8 - Task 3: Stream Dataflow and Process Synchronization
 * Objective: To Create Pipes, Redirect Streams, Execute Connected Commands,
 *           Close Descriptors, Synchronize Processes, Test Data Flow.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    printf("=== Week 8 Task 3: Stream Data Flow & Verification ===\n\n");
    int pipefd[2];
    if (pipe(pipefd) < 0) {
        perror("pipe failed");
        return 1;
    }

    pid_t pid = fork();
    if (pid == 0) {
        // Child: Read from pipe, verify integrity and count bytes
        close(pipefd[1]);
        char buffer[256];
        ssize_t bytes = read(pipefd[0], buffer, sizeof(buffer) - 1);
        if (bytes > 0) {
            buffer[bytes] = '\0';
            printf(" [Consumer Process (PID %d)] Received %zd bytes: '%s'\n",
                getpid(), bytes, buffer);
        }
        close(pipefd[0]);
    }
}

```

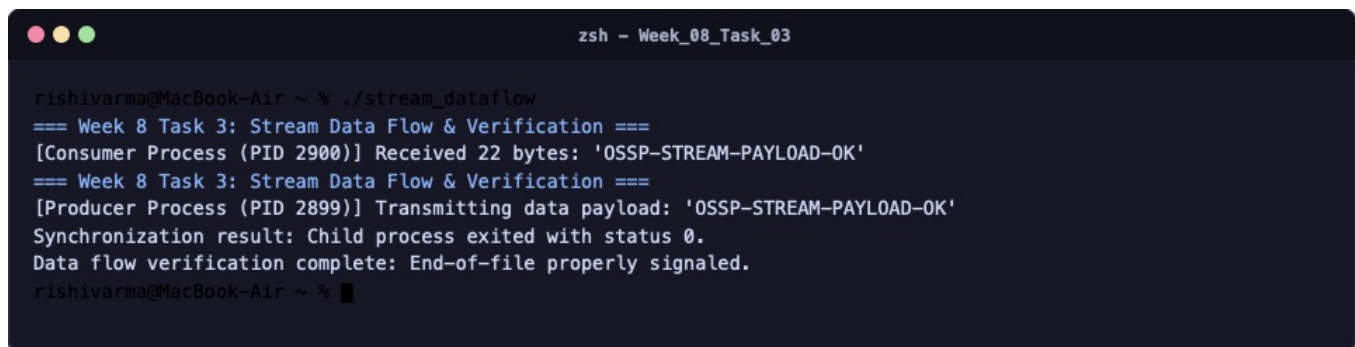
```

        exit(0);
    }

    // Parent: Write structured payload to pipe
    close(pipefd[0]);
    const char *msg = "OSSP-STREAM-PAYLOAD-OK";
    printf(" [Producer Process (PID %d)] Transmitting data payload: '%s'\n", getpid(), msg);
    write(pipefd[1], msg, strlen(msg));
    close(pipefd[1]); // Close write end to trigger EOF on reader

    int status;
    waitpid(pid, &status, 0);
    printf("\nSynchronization result: Child process exited with status %d.\n",
    WEXITSTATUS(status));
    printf("Data flow verification complete: End-of-file properly signaled.\n");
    return 0;
}

```

Output:


```

zsh - Week_08_Task_03

rishivarma@MacBook-Air ~ % ./stream_dataflow
=== Week 8 Task 3: Stream Data Flow & Verification ===
[Consumer Process (PID 2900)] Received 22 bytes: 'OSSP-STREAM-PAYLOAD-OK'
=== Week 8 Task 3: Stream Data Flow & Verification ===
[Producer Process (PID 2899)] Transmitting data payload: 'OSSP-STREAM-PAYLOAD-OK'
Synchronization result: Child process exited with status 0.
Data flow verification complete: End-of-file properly signaled.
rishivarma@MacBook-Air ~ %

```

TASK 4**Objective / Skill:**

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Implementation / Work:

```

/*
 * Week 8 - Task 4: Multi-Stage Pipelines with Descriptor Arrays
 * Objective: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes,
 *             Coordinate Execution, Cleanup Resources, Test Long Pipelines.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CMDS 3

int main() {
    printf("=== Week 8 Task 4: Multi-Stage Pipeline (3 Commands) ===\n");
    printf("Executing pipeline: printf \"gamma\\nalpha\\nbeta\\n\" | sort | grep -n \"[a-z]\"\\n\\n");

    // Command definitions
    char *cmd0[] = {"printf", "gamma\\nalpha\\nbeta\\n", NULL};
    char *cmd1[] = {"sort", NULL};
    char *cmd2[] = {"grep", "-n", "[a-z]", NULL};
    char **cmds[NUM_CMDS] = {cmd0, cmd1, cmd2};

    int pipes[NUM_CMDS - 1][2];
    for (int i = 0; i < NUM_CMDS - 1; i++) {
        if (pipe(pipes[i]) < 0) {
            perror("pipe allocation failed");
            return 1;
        }
    }

    pid_t pids[NUM_CMDS];
    for (int i = 0; i < NUM_CMDS; i++) {
        pids[i] = fork();
        if (pids[i] == 0) {
            // Child process i

```

```

        if (i > 0) dup2(pipes[i - 1][0], STDIN_FILENO);
        if (i < NUM_CMDS - 1) dup2(pipes[i][1], STDOUT_FILENO);

        // Close all descriptor array handles
        for (int j = 0; j < NUM_CMDS - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(cmds[i][0], cmds[i]);
        perror("execvp failed");
        exit(1);
    }
}

// Parent: Close all descriptor array handles
for (int j = 0; j < NUM_CMDS - 1; j++) {
    close(pipes[j][0]);
    close(pipes[j][1]);
}

// Coordinate execution and wait for all children
for (int i = 0; i < NUM_CMDS; i++) {
    int status;
    waitpid(pids[i], &status, 0);
}

printf("\nCoordinated execution of 3-stage pipeline completed with zero descriptor
leaks!\n");
return 0;
}

```

Output:

```

zsh - Week_08_Task_04

rishivarma@MacBook-Air ~ % ./multi_stage_pipe
1:alpha
2:beta
3:gamma
=== Week 8 Task 4: Multi-Stage Pipeline (3 Commands) ===
Executing pipeline: printf "gamma\nalpha\nbeta\n" | sort | grep -n "[a-z]"
Coordinated execution of 3-stage pipeline completed with zero descriptor leaks!
rishivarma@MacBook-Air ~ %

```